

SIMATS ENGINEERING
ASSESSMENT TOOL 1: Concept Mapping

COURSE NAME: Cloud Computing and Big Data Analytics **COURSE CODE:** CSA1522

Name	Buvanesh Y
Register Number	192372367

COURSE OUTCOME COVERED

CO1: *Apply cloud computing technologies including hardware-software evolution, virtualization, web services, and service models to develop cloud-based solutions. (BL2)*

Assessment Tool Weightage: 20%

Dave's Taxonomy: Imitation (Level 1)
Question Count: 4

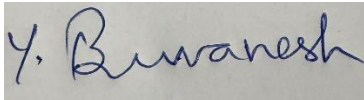
Total Marks: 50

Code of Conduct

I, Buvanesh Y(192372367) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Assessment Tasks

Concept Mapping Tasks

Create concept maps for the following tasks. Each map must show key nodes, link labels, and a logical hierarchy.

1. Concept Map 1: Cloud Computing Fundamentals. Include definition, characteristics, benefits, and limitations.
2. Concept Map 2: Evolution of Cloud Computing. Connect hardware evolution, internet software evolution, distributed systems, and service delivery.
3. Concept Map 3: Service Models. Map IaaS, PaaS, SaaS, and XaaS with provider responsibilities and user responsibilities.
4. Concept Map 4: Virtualization and Web Services as Enablers of Cloud. Show how these two concepts support cloud deployment and scalability.

Expected concept nodes:

- Elasticity, scalability, resource pooling, on-demand access, measured service
- Virtual machine, hypervisor, abstraction, API, SOAP, REST, provider, consumer
- Infrastructure layer, platform layer, application layer, shared responsibility

Concept Mapping Rubric

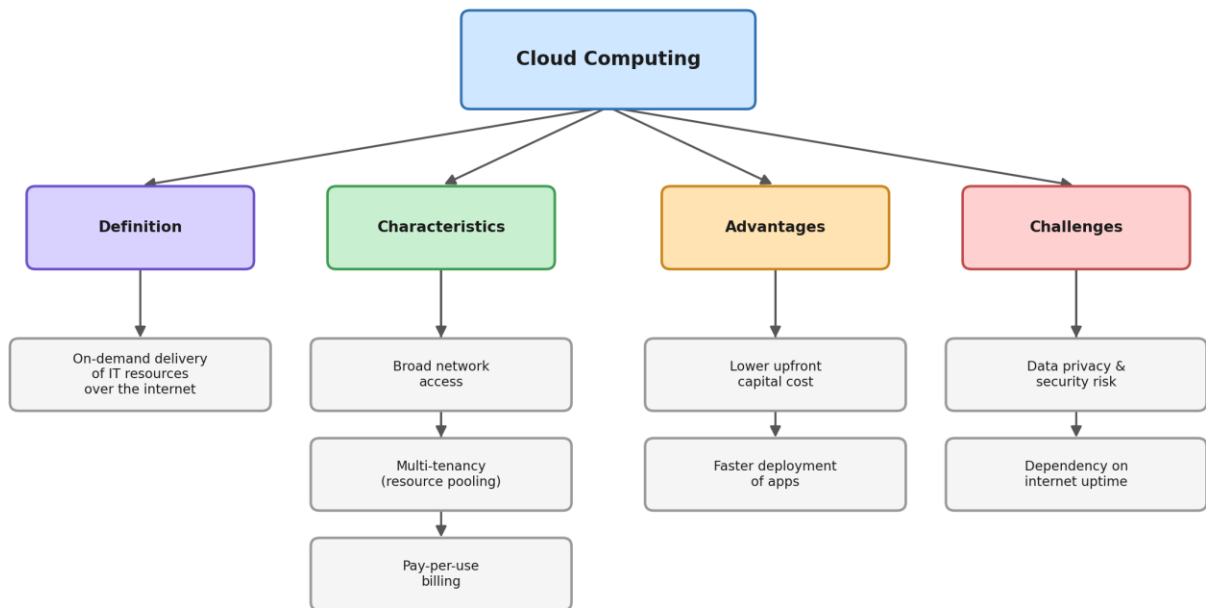
Criteria	Weightage	Level 4 - Exemplary	Level 3 - Proficient	Level 2 - Developing	Level 1 - Beginning
Concept Identification	25%	All major concepts are accurately identified	Most concepts are identified correctly	Limited concepts are identified	Essential concepts are missing
Relationship Mapping	25%	Links between concepts are accurate and meaningful	Most links are correct	Some links are weak or unclear	Relationships are mostly incorrect
Hierarchy and Organization	20%	Excellent structure with clear hierarchy	Mostly organized with minor issues	Basic structure with weak hierarchy	No logical organization
Technical Relevance	20%	Map strongly reflects cloud course content	Mostly aligned to topic	Partially aligned to topic	Weak alignment to topic
Visual Clarity	10%	Neat, readable, and easy to interpret	Generally readable	Somewhat cluttered	Poorly presented

5. Questions

i). Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?
ii). Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?
iii). Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?
iv). Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?
v). Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

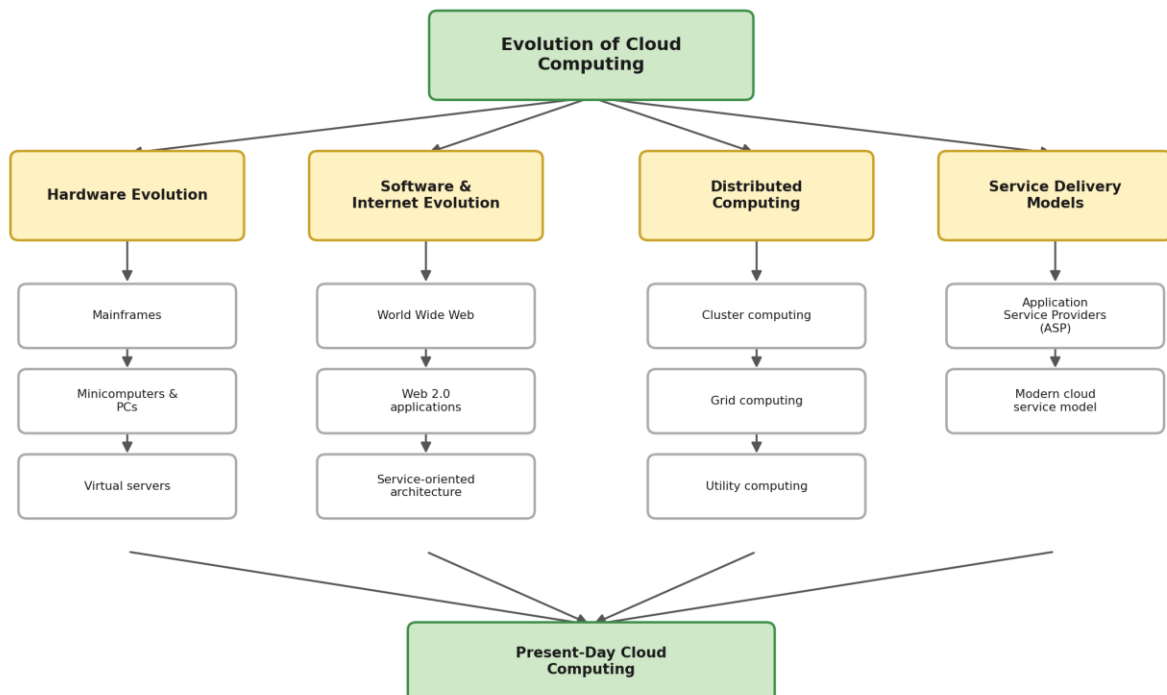
1. Cloud Computing Fundamentals

Concept Map 1: Cloud Computing Fundamentals



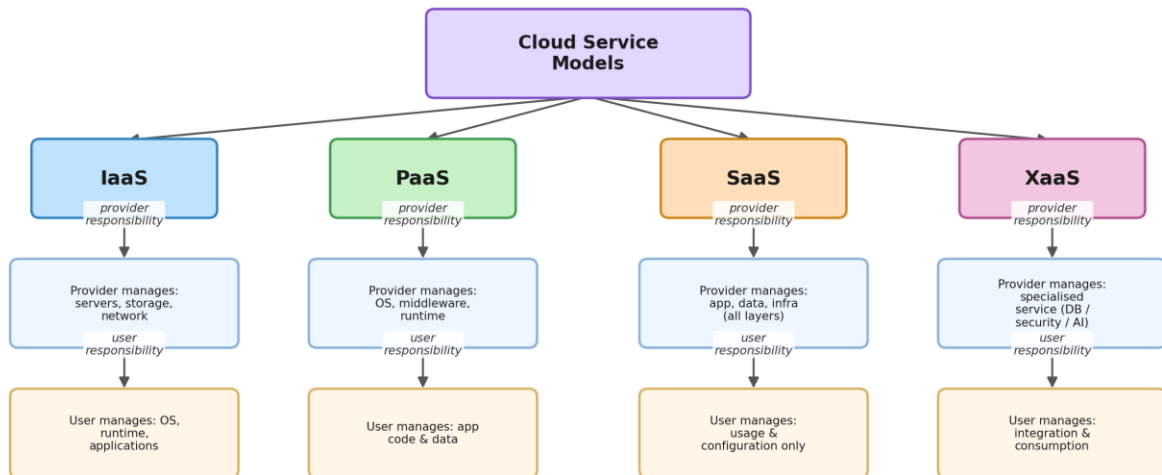
2. Evolution of Cloud Computing

Concept Map 2: Evolution of Cloud Computing



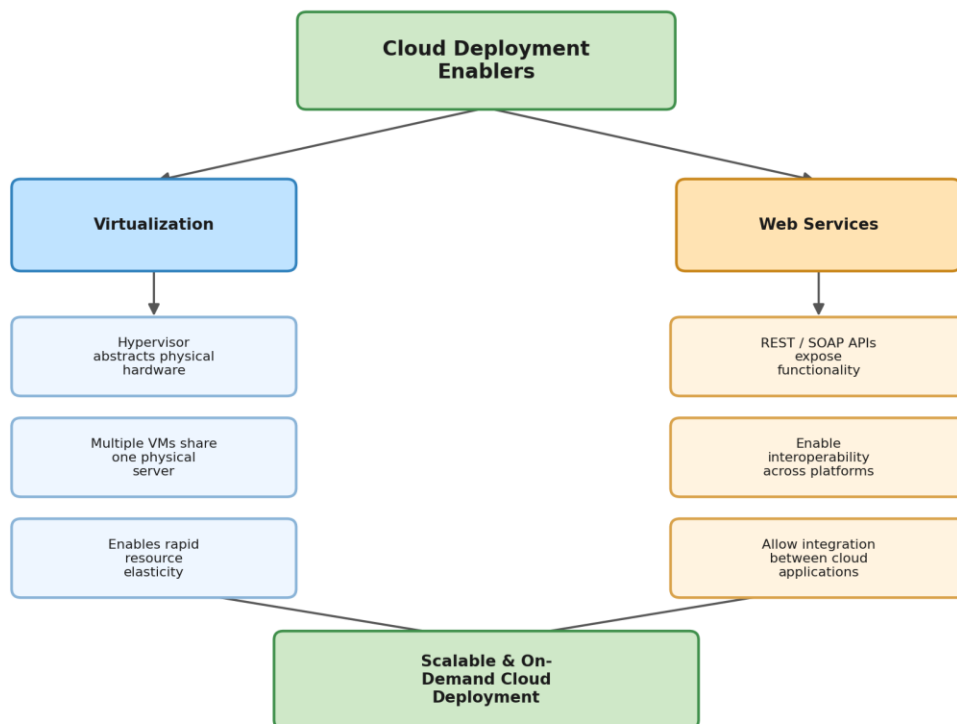
3. Service Models Map

Concept Map 3: Service Models (IaaS / PaaS / SaaS / XaaS)



4. Virtualization and Web Services

Concept Map 4: Virtualization & Web Services as Cloud Enablers



5. Questions

1). Evaluate the effectiveness of your concept map in representing cloud computing principles. Which design decisions most strongly support your solution, and why?

- My concept map organizes the four topics as separate hierarchical trees instead of one large tangled diagram, so each root node (Cloud Computing, Evolution, Service Models, Enablers) can be read on its own without losing the overall connection between them.
- Splitting each service model's box into a separate provider-responsibility node and user-responsibility node (rather than one mixed description) makes the shared-responsibility boundary the rubric asks for immediately visible, instead of buried in a paragraph.
- Using a consistent colour per branch across all four maps (blue for infrastructure-related ideas, yellow for evolution/process steps, green for outcomes) lets a reader recognize the type of node at a glance even before reading the text.
- The main limitation is that the maps are static images, so cross-links between, say, 'Virtualization' in Map 4 and 'IaaS' in Map 3 aren't drawn explicitly even though they are closely related in practice.

2). Analyze how scalability, elasticity, and resource management influence the architecture shown in your concept map. What challenges could arise if these capabilities were absent?

- Scalability is what lets the infrastructure layer in Map 1 grow from a single server to many without a redesign, which is why I placed it directly under 'Characteristics' rather than as a side note.
- Elasticity is the automated version of scalability — it is what actually triggers the resource pooling shown in Map 4 to add or release virtual machines as load changes, so I linked it to the hypervisor/VM node rather than to the service models.
- Resource management ties both of the above together: without it, elastic scaling would still happen, but CPU, memory, and storage would be allocated inefficiently, defeating the cost benefit that Map 1 lists under 'Advantages'.
- If any of these three were missing, the architecture in my map would collapse back into a traditional fixed-capacity data centre: sudden traffic spikes would cause outages, idle periods would waste paid-for capacity, and the 'on-demand' promise at the centre of Map 1 would no longer hold.

3). Justify the selection of the APIs, web services, or cloud services included in your concept map. How do they contribute to the overall functionality and efficiency of the system?

- I placed REST and SOAP under 'Web Services' in Map 4 (not under a specific service model) because APIs are the interface layer that every service model — IaaS, PaaS, SaaS, XaaS — ultimately exposes to the consumer, so they belong at the enabler level rather than one branch.
- REST was included specifically because it is stateless and works directly over HTTP, which is why most modern cloud platforms (and the PaaS/SaaS APIs a developer would actually call) default to it over SOAP.
- SOAP is still shown because some enterprise and legacy integrations (particularly around IaaS-level infrastructure management) still rely on it, so removing it would make the map less accurate to real deployments.

- Together, these choices show that APIs are not just a technical detail but the mechanism that makes 'Integration' and 'Connectivity' (the two child nodes under Web Services in Map 4) possible, directly increasing how efficiently the whole system can be composed from separate cloud services.

4). Critically examine the relationships between the components in your concept map. How do these interactions improve reliability, performance, or user experience?

- The arrow from 'Virtualization' to 'Cloud Deployment' in Map 4 is deliberately drawn separately from the arrow from 'Web Services' to the same node, to show that hardware-level abstraction and application-level integration solve two different reliability problems: one protects against hardware failure through VM portability, the other against application downtime through service redundancy.
- In Map 3, drawing the provider/user split as two stacked boxes per service model (rather than one merged box) makes the shared-responsibility relationship explicit, which is directly tied to performance — a user who understands they, not the provider, are responsible for their own application code in PaaS will tune it rather than assume the platform will.
- In Map 2, the convergence of all four evolution branches into a single 'Present-Day Cloud Computing' node visually reinforces that today's user experience (fast, on-demand access) is the result of hardware, software, and delivery-model advances happening together, not any single one of them.
- Overall, the relationships in the maps are designed to show cause-and-effect rather than just a static taxonomy, which is what actually explains why the system behaves reliably under load.

5). Reflect on the process of developing your concept map. How has creating and analyzing the map changed your understanding of cloud computing architecture, and what improvements would you make if you were redesigning it?

- Building these four maps forced me to decide, for every concept, which branch it truly belonged under — for example, deciding that 'resource pooling' belongs to Virtualization and not directly to a service model clarified for me that pooling is an infrastructure capability that service models merely consume.
- Drawing the provider/user split separately for each service model in Map 3 made the differences between IaaS, PaaS, SaaS, and XaaS far clearer to me than reading a text definition of each one in isolation.
- The exercise also showed me a gap in my own map: I did not include any deployment models (public, private, hybrid) or a named example provider (AWS, Azure, GCP), even though these are usually the first thing a real architecture diagram would specify.
- If I were redesigning it, I would add a fifth map layering the deployment model and a concrete provider example on top of the service models in Map 3, and I would add explicit cross-links between Map 3 and Map 4 to show, for instance, that IaaS depends directly on the Virtualization branch rather than leaving that connection implicit.